

PRAKTIKUM SISTEM OPERASI

MODUL 15
(Keamanan Linux)



Oleh:

(Ammar Dzaki Nandana)

(2311104071)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026

MODUL 15

Pendahuluan

Pada praktikum Modul 15 dipelajari berbagai konsep dasar keamanan pada sistem operasi Linux. Materi yang dibahas meliputi **integritas data menggunakan hashing, Avalanche Effect, pengaruh metadata terhadap nilai hash, konfidensialitas data menggunakan EncFS, serta enkripsi dan dekripsi menggunakan GPG (GNU Privacy Guard)**. Praktikum ini bertujuan untuk memahami bagaimana keamanan data diterapkan melalui proses verifikasi integritas, penyimpanan terenkripsi, serta pertukaran data secara aman.

1. Integritas Data Menggunakan Hashing

Pada percobaan pertama dilakukan pengujian integritas file menggunakan algoritma hashing. File yang digunakan adalah file sistem `/etc/passwd`. Nilai hash dihitung menggunakan tiga algoritma yang berbeda, yaitu **MD5, SHA-256, dan SHA-512**.

Menghitung Hash File `/etc/passwd`

```
md5sum /etc/passwd
sha256sum /etc/passwd
sha512sum /etc/passwd
```

Selanjutnya file tersebut disalin ke direktori home dengan nama **test_0.txt** menggunakan perintah:

```
cp /etc/passwd ~/test_0.txt
```

Untuk memastikan file berhasil disalin, dilakukan pengecekan isi direktori home menggunakan:

```
ls ~
```

Kemudian hash file hasil salinan dihitung kembali menggunakan perintah:

```
md5sum ~/test_0.txt
sha256sum ~/test_0.txt
sha512sum ~/test_0.txt
```

Selanjutnya nama file diubah menggunakan perintah:

```
mv ~/test_0.txt ~/file_0.txt
```

Setelah proses *rename*, nilai hash dihitung kembali:

```
md5sum ~/file_0.txt
sha256sum ~/file_0.txt
sha512sum ~/file_0.txt
```

Analisis

Hasil pengujian menunjukkan bahwa nilai hash pada ketiga file tetap sama meskipun nama file telah diubah. Hal ini terjadi karena fungsi hash hanya menghitung isi file, bukan nama file maupun lokasi penyimpanannya.

Selama isi file tidak berubah, nilai hash akan selalu sama. Percobaan ini menunjukkan bahwa hashing merupakan metode yang efektif untuk memverifikasi integritas suatu file.

2. Avalanche Effect

Percobaan kedua bertujuan untuk membuktikan sifat **Avalanche Effect**, yaitu perubahan yang sangat kecil pada isi data akan menghasilkan perubahan hash yang sangat besar.

Langkah pertama adalah mengunduh file uji menggunakan perintah:

```
wget https://tiny.cc/test1_txt -O test_1.txt
```

Kemudian dihitung nilai hash awal menggunakan:

```
md5sum ~/test_1.txt  
sha256sum ~/test_1.txt  
sha512sum ~/test_1.txt
```

Selanjutnya isi bagian akhir file diperiksa menggunakan:

```
tail ~/test_1.txt
```

File kemudian dibuka menggunakan editor **Nano**:

```
nano ~/test_1.txt
```

Pada editor dilakukan langkah berikut:

1. Pindah ke bagian akhir file.
2. Menghapus satu karakter berupa tanda titik (.).
3. Menyimpan perubahan menggunakan:

```
Ctrl + X  
Y  
Enter
```

Setelah file disimpan, hash dihitung kembali menggunakan:

```
md5sum ~/test_1.txt  
sha256sum ~/test_1.txt  
sha512sum ~/test_1.txt
```

Analisis

Walaupun perubahan yang dilakukan hanya menghapus satu karakter pada akhir file, seluruh nilai hash berubah secara signifikan. Fenomena ini disebut **Avalanche Effect**, yaitu karakteristik fungsi hash di mana perubahan yang sangat kecil pada data akan menghasilkan nilai hash yang sama sekali berbeda. Sifat ini menjadikan algoritma hash sangat baik digunakan untuk mendeteksi perubahan data sekecil apa pun.

3. Metadata pada File DOC

Percobaan berikutnya bertujuan membuktikan bahwa perubahan metadata juga dapat memengaruhi nilai hash suatu file.

Langkah pertama adalah mengunduh file **test_1.doc** kemudian menghitung hash awal menggunakan:

```
md5sum ~/test_1.doc
sha256sum ~/test_1.doc
sha512sum ~/test_1.doc
```

File kemudian dibuka menggunakan LibreOffice:

```
libreoffice ~/test_1.doc
```

Selanjutnya dilakukan langkah berikut:

1. Menambahkan tulisan:

```
abcdef
```

1. Menyimpan file menggunakan **Ctrl + S**.
2. Memilih **Keep Current Format**.
3. Menghapus kembali tulisan **abcdef**.
4. Menyimpan kembali file.

Setelah proses tersebut selesai, hash file dihitung kembali menggunakan:

```
md5sum ~/test_1.doc
sha256sum ~/test_1.doc
sha512sum ~/test_1.doc
```

Analisis

Walaupun isi teks pada dokumen kembali sama seperti semula, nilai hash tetap berubah. Hal ini disebabkan karena file DOC menyimpan **metadata**, seperti waktu modifikasi, jumlah penyimpanan (*save count*), serta informasi revisi dokumen. Metadata tersebut berubah setiap kali file disimpan sehingga menghasilkan nilai hash yang berbeda. Percobaan ini menunjukkan bahwa hashing menghitung keseluruhan isi file, termasuk metadata yang tersimpan di dalamnya.

4. Konfidensialitas Data Menggunakan EncFS

Pada percobaan ini dipelajari penggunaan **EncFS (Encrypted File System)** untuk membuat direktori terenkripsi.

Langkah pertama adalah menginstal EncFS menggunakan:

```
sudo apt update
sudo apt install encfs
```

Selanjutnya dibuat dua direktori:

```
mkdir -p ~/modul15/enc ~/modul15/dec
```

Kemudian dijalankan EncFS:

```
encfs ~/modul15/enc ~/modul15/dec
```

Saat pertama kali dijalankan dipilih opsi:

```
y
```

Kemudian dibuat password untuk proses enkripsi.

Selanjutnya tiga file sistem disalin ke folder hasil dekripsi:

```
cp /etc/passwd /etc/hostname /etc/os-release ~/modul15/dec/
```

Isi direktori terenkripsi kemudian diperiksa menggunakan:

```
ls -la ~/modul15/enc/
```

Terlihat bahwa nama file berubah menjadi karakter acak karena telah terenkripsi.

Selanjutnya dilakukan penghapusan salah satu file terenkripsi.

Melihat isi folder:

```
ls ~/modul15/enc
```

Menghapus salah satu file:

```
rm ~/modul15/enc/2XcPkYIS-S5-jWz1S2DW3L0X
```

Kemudian dilakukan pengecekan:

```
ls ~/modul15/dec
```

Terlihat bahwa file yang bersesuaian pada folder hasil dekripsi juga ikut hilang.

Setelah selesai dilakukan proses **unmount**:

```
fusermount -u ~/modul15/dec
```

Kemudian diperiksa menggunakan:

```
ls -la ~/modul15/dec
```

```
ls -la ~/modul15/enc
```

Selanjutnya dilakukan proses **mount** kembali pada folder lain.

Membuat folder baru:

```
mkdir ~/modul15/sembarang
```

Melakukan mount:

```
encfs ~/modul15/enc ~/modul15/sembarang
```

Kemudian memasukkan password yang sama sehingga seluruh file dapat dibaca kembali secara normal.

Analisis

Percobaan ini menunjukkan bahwa EncFS mengenkripsi file secara transparan. Folder terenkripsi hanya menampilkan nama file acak sehingga isi data tidak dapat dibaca secara langsung. Selama pengguna melakukan mount menggunakan password yang benar, seluruh file akan tampil kembali dalam bentuk normal. Dengan demikian EncFS mampu menjaga **kerahasiaan (confidentiality)** data yang disimpan pada sistem Linux.

5. GPG (GNU Privacy Guard)

Pada bagian terakhir dipelajari penggunaan **GNU Privacy Guard (GPG)** untuk membuat pasangan kunci (*key pair*), mengekspor public key, mengimpor public key milik pengguna lain, serta melakukan enkripsi dan dekripsi pesan.

Langkah pertama adalah menginstal GPG:

```
sudo apt update  
sudo apt install gnupg
```

Selanjutnya dibuat pasangan kunci menggunakan:

```
gpg --gen-key
```

Selama proses pembuatan key diisi informasi seperti:

```
Nama : Candra Dinata  
Email : test1@gmail.com  
Password : *****
```

Untuk melihat daftar key yang telah dibuat digunakan perintah:

```
gpg --list-keys
```

Sedangkan untuk melihat fingerprint digunakan:

```
gpg --fingerprint test1@gmail.com
```

Selanjutnya public key diekspor menggunakan:

```
gpg --armor --export test1@gmail.com > mypublic_key.asc
```

Kemudian nama file diubah menjadi:

```
mv mypublic_key.asc 2311104061.asc
```

Apabila menerima public key dari pengguna lain, proses import dilakukan menggunakan:

```
gpg --import 2311104061.asc
```

Untuk melakukan enkripsi pesan, terlebih dahulu dibuat file:

```
nano pesan.txt
```

Isi file:

```
Halo teman, ini pesan rahasia jangan diberi tahu.
```

Kemudian file dienkripsi menggunakan:

```
gpg --encrypt --armor -r email_teman@gmail.com pesan.txt
```

Hasil proses enkripsi berupa file:

```
pesan.txt.asc
```

Untuk membaca pesan tersebut dilakukan dekripsi menggunakan:

```
gpg --decrypt pesan.txt.asc
```

atau menyimpan hasil dekripsi ke dalam file:

```
gpg --output hasil.txt --decrypt pesan.txt.asc
```

Analisis

GPG menggunakan mekanisme **kriptografi kunci publik (public key cryptography)**. Pengguna dapat membagikan public key kepada orang lain untuk menerima pesan terenkripsi, sedangkan private key tetap disimpan secara rahasia untuk melakukan dekripsi. Dengan mekanisme ini, hanya pemilik private key yang dapat membaca isi pesan sehingga kerahasiaan dan keamanan komunikasi dapat terjaga.

Kesimpulan

Pada praktikum Modul 15 telah dipelajari berbagai konsep keamanan pada sistem operasi Linux, meliputi verifikasi integritas file menggunakan algoritma hash, pembuktian sifat **Avalanche Effect**, pengaruh metadata terhadap perubahan nilai hash, implementasi sistem berkas terenkripsi menggunakan **EncFS**, serta penggunaan **GNU Privacy Guard (GPG)** untuk proses enkripsi dan dekripsi data. Melalui praktikum ini dapat dipahami bahwa keamanan informasi tidak hanya bergantung pada kerahasiaan data, tetapi juga mencakup aspek integritas dan autentikasi. Berbagai mekanisme tersebut merupakan bagian penting dalam menjaga keamanan data pada sistem operasi Linux.

